

The synapse that never fills up

Live defence & judge Q&A playbook

A fixed-size synaptic weight matrix can process a sequence of unbounded duration without allocating a new slot per token—but retrieval degrades through interference between stored associations.

ROUND

12 September 2026 · 8:00 AM–6:00 PM IST

CORE DEMO

3 minutes + judge questions

VERIFIED SEED

42 · deterministic stress case included

ARTIFACT

sujalmeena7.github.io/synapse-never-fills-up

Room rule: never guess a number. Predict, measure, then explain.

1. What you must make the judges remember

The precise central claim

A fixed-size synaptic weight matrix can process a sequence of unbounded duration without allocating a new slot per token—but retrieval degrades through interference between stored associations.

This does **not** mean infinite perfect retention. “Unbounded duration” describes how long a sequence the recurrent update can process without growing the state. Capacity remains finite because crosstalk accumulates.

Your contribution in one sentence

We turned a frontier architectural mechanism into a visible, falsifiable learning experience: the learner can grow the sequence, inspect the fixed state, force recall failure, and see exactly which memories caused it.

Why this is more than a visualisation

- The matrix, recall, crosstalk bars, decode scores, and curves are computed live.
- The learner changes real variables: write count, memory load, cue, key regime, and sparsity.
- Ground truth sits beside the memory’s estimate.
- A failure case is part of the lesson rather than hidden.
- BDH is woven into the mechanism; it is not a paragraph added at the end.

2. Three-minute presentation script

Use this if no presentation duration is provided. It leaves room for questions.

0:00–0:25 — Problem and claim

Screen: hero.

“Transformers remember earlier tokens using a KV cache that grows with the sequence. We ask whether working memory can instead live in fixed-size synaptic state. Our claim is precise: fixed state can process an unbounded-duration sequence without a per-token slot, but retrieval still degrades through interference.”

0:25–0:50 — Mechanism

Screen: *This is a memory.*

“Each association is written through a Hebbian outer product: $S \leftarrow \lambda S + kv^T$. A cue reads the same matrix with $r = S^T q$. Scrubbing the writes changes every visible cell, but the matrix remains 32 by 32.”

Move **writes applied** from 0 → 8. Point to **1024 scalars · constant**.

0:50–1:20 — Fixed state versus KV cache

Screen: *Add more memories.*

Move **pairs stored** from 8 → 64.

"The fast-weight state remains 1024 scalars. The illustrative KV footprint grows from 512 to 4096. At short sequences the matrix can even be larger; the claim is not 'always cheaper.' The claim is that its state does not grow with sequence length."

1:20–2:05 — Reveal the failure mechanism

Screen: *Read one memory back and What did the memory actually say?*

At seed 42, set **pairs stored = 24**, cue **ARIA** → **0**.

"With six pairs the cue is recalled correctly. At 24 pairs, ARIA is a deterministic stress case: the expected digit is 0, but accumulated overlap makes the memory choose 3. Nothing was deleted. The green self-signal remains; the red bars are measured dot-product leakage from every other key."

Point to the expected answer and memory pick. This is your 60-second learning moment: **failure through interference, not eviction**.

2:05–2:40 — Connect to BDH

Screen: *Where this lives in Dragon Hatchling.*

Toggle **dense signed** → **sparse non-negative**, start at **5% active**, then show **50% active**.

"BDH reports sparse, non-negative activation. Non-negativity removes sign cancellation, so collisions only add. Sparsity makes collisions rare enough to keep that viable. Our toy does not run a BDH checkpoint; it isolates and measures this mechanism. At 50% active our toy collapses, making the need for sparsity visible."

2:40–3:00 — Limitation and close

"The limitation is equally important: this is temporary within-session state, not durable cross-session learning, and our keys are random rather than learned. The reusable contribution is a live substrate in which a learner can reproduce both the benefit and the failure of fixed-state synaptic memory in under a minute."

Stop. Do not keep scrolling. Invite questions.

3. If you receive five minutes

Use the same three-minute flow, then add:

1. **30 s — derivation:** show how the unnormalised attention sum reassociates.
2. **30 s — prediction:** ask a judge to predict what happens if `d_k` grows.
3. **30 s — evidence:** run `npm run verify` or show `EVIDENCE.md`.
4. **30 s — recent landscape:** BDH (2025), BDH-CQ (2026), and Gated DeltaNet (2024/ICLR 2025) demonstrate that recurrent matrix memory is an active direction, not a historical curiosity.

If time is cut, omit the derivation—not the live failure or limitation.

4. Derive the mechanism correctly

Start with softmax attention

```
out_t = \sum_i \text{softmax}(q_t \cdot k_i) v_i
```

This requires access to previous keys and values. **You cannot simply reassociate softmax attention into one matrix** because its query-dependent normalisation couples the terms.

Change the mechanism to the unnormalised linear/fast-weight special case

```
out_t = \sum_{i \leq t} (q_t \cdot k_i) v_i
      = (\sum_{i \leq t} v_i k_i^T) q_t
      = S_t^T q_t
```

The artifact writes S in $d_k \times d_v$ orientation:

```
WRITE  S_t = \lambda S_{t-1} + k_t v_t^T
READ   r_t = S_t^T q_t
```

This is a **different attention mechanism**, not an exact replacement preserving softmax outputs. Modern linear attention often uses feature maps and a normalising denominator; the artifact intentionally uses the simplest unnormalised associative case so every state variable remains visible.

Interference decomposition

For a cue equal to stored unit-norm key k_j :

```
r = S^T k_j
  = v_j (k_j \cdot k_j)          signal = 1 \cdot v_j
  + \sum_{i \neq j} v_i (k_i \cdot k_j)  crosstalk
```

No slot fills up and no item is evicted. Recall fails when projected crosstalk changes the top-scoring value.

What “capacity” means in this toy

The toy decodes against a **10-symbol value vocabulary** and reports top-1 symbol recall. It does not measure exact retrieval of an arbitrary unique vector for every item. The experiment teaches how key overlap affects associative class retrieval; it is not a universal capacity benchmark for BDH or all fast-weight memories.

5. Trace the code if a judge asks

```
index.html
├─ src/app.js      controls, corpus construction, canvas drawing
├─ src/engine.js   all memory mathematics; no DOM

src/motion.js      cosmetic only: reveal, particles, spring counters
vendor/motion.js   vendored Motion library; no runtime CDN dependency
scripts/verify.mjs deterministic acceptance tests at seed 42
scripts/smoke.mjs  DOM, controls, and reveal regression test
EVIDENCE.md        claim → acceptance test → result → limitation
```

One interaction end-to-end

When **pairs stored** changes:

1. `app.js` reads the slider.
2. It constructs deterministic keys/values from the displayed seed.
3. `engine.js` allocates `S` once and applies every outer-product write.
4. `engine.js` probes each cue and calculates recall/margins.
5. `app.js` draws the results and places ground truth beside the estimate.
6. `motion.js` only animates the number; it cannot alter the result.

Deleting `motion.js` changes presentation, not mathematics.

6. Predict changes before touching the controls

Change	Prediction	Reason
pairs $8 \rightarrow 64$	recall eventually falls; <code>S</code> stays fixed	more crosstalk terms; one allocation
<code>d_k</code> $32 \rightarrow 128$	average recall rises at fixed load	random keys are closer to orthogonal in higher dimension
λ $1.0 \rightarrow 0.9$	old pairs fade; recent pairs dominate	earlier writes are geometrically down-weighted
non-negative activity $5\% \rightarrow 50\%$	recall falls sharply	overlaps cannot cancel and become frequent
new seed	individual misses change; aggregate trend persists	keys change, scaling pressure does not
learned instead of random keys	could improve capacity	training can reduce harmful overlap, but this repo does not test it

The sparsity scaling argument—state it as a heuristic

For random unit-norm dense signed keys in `d` dimensions:

```
E[k_i · k_j] ≈ 0
E|k_i · k_j| ≈ √(2/π) / √d
aggregate signed noise scales roughly as √(m/d)
```

For random sparse non-negative keys with `a` active units:

```
E[k_i · k_j] ≈ a/d
aggregate overlap scales roughly as m·a/d because signs cannot cancel
```

Equating the rough scales suggests `a ≈ √(d/m)` as an **expected-interference crossover heuristic**. It is not a theorem guaranteeing a particular recall rate. The verifier checks the overlap prediction empirically.

7. Verified numbers worth knowing

All are deterministic outputs of `npm run verify` at seed 42.

Test	Result
State size, <code>d_k=d_v=32</code>	1024 scalars at every sequence length
KV footprint	<code>64m</code> ; equals state at <code>m=16</code> , then keeps growing
Dense recall vs load	100% at <code>m≤12</code> ; 99.5% at 16; 89.8% at 32; 77.4% at 48
16-pair recall vs <code>d_k</code>	70.9% at 8; 91.4% at 16; 99.5% at 32; 100% at ≥64
Dense, <code>d=256, m=16</code>	100% recall; margin 0.689
Non-negative, 50% active	47.0% recall; margin -0.033
Non-negative, 5% active	100% recall; margin 0.612
Dense overlap	measured 0.0508; predicted 0.0499
Sparse overlap, <code>a=6</code>	measured 0.0226; predicted 0.0234

Do not imply the fixed matrix is always smaller. At `m<16` in this particular configuration, the illustrative KV footprint is smaller. The architectural advantage is asymptotic state growth, not universal superiority.

8. Likely judge questions

“Is this just Mamba or another SSM?”

No. The artifact demonstrates a recurrent fast-weight matrix written by outer products. Mamba uses selective state-space dynamics. BDH-GPU is described using ReLU-low-rank transformations and linear attention; the challenge explicitly warns not to classify BDH as a Mamba-style SSM.

“Did you run or reproduce BDH?”

No. The challenge does not require an unavailable checkpoint. Our BDH module is grounded in published equations, reported architectural properties, and clearly labelled toy computation. All numbers generated by this repository describe the toy, not a Pathway checkpoint.

“Then what does this prove about BDH?”

It does not prove BDH’s benchmark performance. It makes one mechanism and design pressure testable: fixed recurrent association trades per-token storage for interference-limited retrieval; sparse non-negative codes alter that interference. BDH is the recent case study connecting the mechanism to a full architecture.

“Does sparse non-negative memory beat dense signed memory?”

Not universally, and not at the 5% setting in this random-key trial if margin is the metric: dense margin is 0.689 versus 0.612. The defensible result is that high-density non-negative codes perform badly because collisions never cancel; sparsity restores viability. That is narrower than “sparse always wins.”

“Why not include normalisation?”

Normalised kernel attention is important in production models, but it adds a second accumulated state and obscures the signal-plus-crosstalk equation. We chose the unnormalised special case because the education objective is to make interference directly visible. The limitation is stated; extending the same substrate with a normaliser is future work.

“Is the memory really unbounded?”

The **input duration** is unbounded with respect to state allocation; retention capacity is not. The matrix can accept another write without growing, but recall quality falls as associations interfere. That distinction is the central lesson.

“Why only ten values?”

A small fixed vocabulary keeps ground truth and competing answers visible. It measures top-1 symbol recall, not exact unique-vector capacity. A larger or unique-value experiment is a valid extension, not a result we claim here.

“What is live versus precomputed?”

Every matrix, curve, bar, decode, and numerical verdict is computed live from `engine.js`. Nothing scientific is precomputed. The neural particles and scroll reveals are cosmetic. Multi-seed curves are recomputed in-browser.

“How do we reproduce it?”

`npm run verify` imports the same engine as the webpage and prints all quoted results. `npm run smoke` loads the real HTML, exercises controls, and asserts no reveal stays hidden. The artifact itself needs no installation or backend.

“Where is the recent research context?”

- **BDH** (2025): synaptic memory and sparse non-negative activation in a post-Transformer architecture.
- **BDH-CQ** (2026): recurrent contextual memory and latent reasoning.
- **Gated DeltaNet** (2024; ICLR 2025): independent gated delta-rule recurrent matrix memory addressing interference and retrieval.
- **Transformers are RNNs** (2020): foundational background, explicitly outside the track’s required 2022–2026 window.

“What would you build next?”

1. Add normalised kernel attention as a side-by-side mechanism.
2. Replace random keys with learned keys and compare overlap distributions.
3. Add unique-value vector retrieval and capacity curves with confidence bands.
4. Run a controlled learner study: explanation accuracy before versus after the 60-second interaction.

9. Evidence labels and forbidden overclaims

Never say:

1. BDH is a Mamba-style SSM.
2. This toy is an official BDH model.
3. The state perfectly remembers an infinite sequence.
4. Sparse representations always beat dense representations.
5. Within-session state is durable cross-session learning.
6. Pathway-reported benchmarks are independent reproductions or deployments.

If benchmark figures arise, say:

“Pathway reports approximately 97.4% top-1 solve rate on its Sudoku Extreme evaluation and 29.5% pass@2 for BDH-CQ on the public ARC-AGI-1 evaluation set. We cite these as developer-reported benchmarks; this repository did not reproduce them.”

10. Day-of checklist

The evening before

- ☐ Read this document once; do not memorise every sentence.
- ☐ Rehearse the three-minute flow twice with a timer.
- ☐ Record the deterministic stress case: seed 42, load 24, ARIA → expected 0, memory pick 3.
- ☐ Run `npm run check`.
- ☐ Open the live artifact with **Ctrl+F5** and verify the precise “process a sequence of unbounded duration” wording.
- ☐ Keep a local server fallback ready: `npm run serve`.
- ☐ Download/offline-copy the repository and `DEFENCE.pdf`.
- ☐ Charge laptop, carry charger, mouse, HDMI/USB-C adapter, and hotspot.

Ten minutes before

- ☐ Close notifications, mail, WhatsApp, and unrelated tabs.
- ☐ Set browser zoom to 110–125% if projected text is small.
- ☐ Open only: live artifact, repository `src/engine.js`, and terminal.
- ☐ Reset artifact seed to 42 and controls to defaults.
- ☐ Test screen sharing and audio if presenting virtually.
- ☐ Keep the local URL ready in a second tab.

During questions

- Answer the question first, then explain.
- Distinguish “our measurement,” “paper-reported,” and “hypothesis.”
- Predict before moving a control.
- If uncertain: “I have not measured that; here is how I would test it.”
- Do not apologise for the toy—explain why visibility required simplification.

11. Emergency fallback

If the hosted page fails:

```
npm run serve
# open http://localhost:8123
```

If animation fails, the mathematics still works and the page force-reveals content. If the browser fails, run:

```
npm run verify
```

Then explain the printed state-size, recall, dimension, sparsity, and overlap results. The numbers—not the motion effects—are the submission.

Ten-second closing

“Our contribution is not another claim that fixed-state memory is free. It is a substrate that lets a learner see both sides: constant allocation and interference-limited recall, connected honestly to BDH and the current fast-weight research landscape.”